

Internet Download Manager (IDM): Complete Technical Architecture & System Specification

An Exhaustive Reverse-Engineering Analysis, Protocol Specification, and Implementation Blueprint for Building the Next-Generation Multi-Stream Downloader "OmniFetch"

DOCUMENT VERSION v1.0.0 (Comprehensive)	REFERENCE SYSTEM Internet Download Manager (IDM v6.4x)	TARGET ARCHITECTURE OmniFetch Core Engine	CLASSIFICATION Engineering Blueprint / Internal
--	--	--	--

EXECUTIVE SUMMARY & ENGINEERING PURPOSE

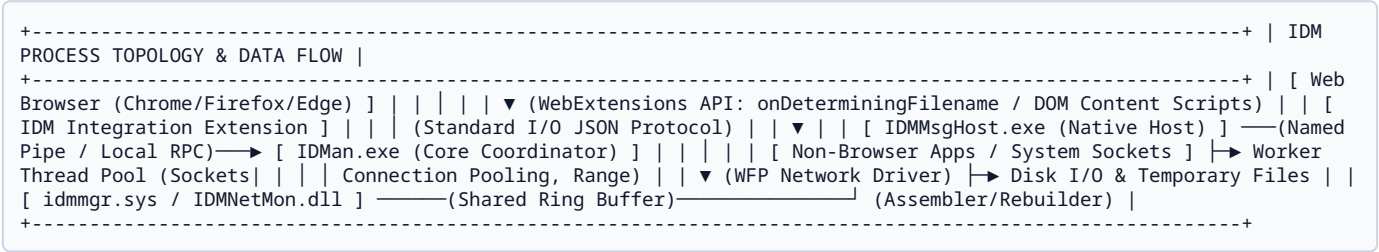
This technical document serves as the foundational architectural blueprint for developing **OmniFetch**, a next-generation download acceleration platform designed to replicate, optimize, and modernize the capabilities of Internet Download Manager (Tonec Inc.). It details every functional and low-level mechanism of IDM—including dynamic multipart segmentation, resilient resume states, dual-tier browser stream interception, video demuxing/muxing, recursive site spiders, network traffic shaping, and OS-level driver integration—while establishing how OmniFetch can systematically transcend IDM's legacy 32-bit Win32 architectural bottlenecks using zero-copy sparse file allocation, asynchronous I/O completion ports, native HTTP/3 (QUIC) multiplexing, and cross-platform runtimes.

1. System Topology & Process Architecture

Internet Download Manager's operational footprint is partitioned across multiple inter-communicating subsystems, native system drivers, and browser-embedded modules. Understanding this segregation is critical for structuring OmniFetch's core daemon and client architecture.

Component Name	Binary / Driver Target	Execution Context	Functional Role & IPC Channel
Core UI & State Dispatcher	IDMan.exe	User Space (Win32 GUI)	Hosts the Win32 message pump, download queue coordinator, segment scheduler, persistent catalog database, and user preferences. Communicates with helpers via Windows Named Pipes and shared memory.
Native Messaging Host	IDMMsgHost.exe	User Space (Console Process)	Bridges Chromium/Gecko browser extensions to IDMan.exe via standard I/O (stdin/stdout) length-prefixed JSON messaging. Forwards download triggers, session cookies, and DOM context.
Elevated Helper & Hook Agent	IDMGrHlp.exe	User Space (System/Tray)	Maintains OS shell hooks, handles system tray notifications, coordinates wake timers, and monitors process termination for auto-restart and crash resilience.
Network Monitor Filter	IDMNetMon.dll	Injected DLL / LSP / WinINet	Hooks system networking APIs (historically Winsock Layered Service Provider, now Windows Filtering Platform Callouts and WinINet/WinHTTP hooks) to intercept raw HTTP/HTTPS traffic system-wide.
Kernel Driver Subsystem	idmmgr.sys	Kernel Mode (Ring 0 Driver)	Windows Filtering Platform (WFP) network driver. Performs deep packet inspection, non-browser link interception, and granular TCP stream redirection without breaking TLS trust stores.

Component Name	Binary / Driver Target	Execution Context	Functional Role & IPC Channel
Browser Extension Module	IDM Integration Module	Browser Context (V2/V3 Extension)	Injects DOM overlays (Floating Video Panel), monitors anchor clicks, intercepts HTTP response headers (webRequest / declarativeNetRequest), and captures dynamic session tokens.



2. The Dynamic File Segmentation Engine

The core computational advantage of IDM over rudimentary download utilities (such as standard single-stream cURL or legacy static multi-part clients) is its **Dynamic Segmentation Algorithm**. Traditional multipart downloaders split a file into N static, immutable chunks prior to initiating downloads (e.g., dividing a 100 MB file into four rigid 25 MB chunks). If one connection encounters server-side TCP throttling or route-specific packet drops, that connection becomes a severe bottleneck ("straggler connection"), while other finished connections sit idle.

2.1 The Dynamic Bisection (Half-Splitting) Mechanics

IDM overcomes this by treating the target file as a continuous byte spectrum $[0, L - 1]$, where L is the **Content-Length** discovered during the initial HTTP handshake:

- Handshake & Range Probing:** The engine issues an initial request with `Range: bytes=0-` or a lightweight probe. It inspects the server's response headers:
 - `Accept-Ranges: bytes`: Confirms byte-range support.
 - `Content-Length: L`: Establishes total byte size.
 - `ETag` & `Last-Modified`: Cached as unique resource fingerprints.
 - `HTTP/1.1 206 Partial Content`: Validates range-slicing capability. If `HTTP/1.1 200 OK` is returned, IDM aborts segmentation and falls back to single-stream downloading.
- Initial Partitioning:** Connection 0 begins consuming bytes sequentially starting from byte offset 0. Simultaneously, if the configured maximum connections $N_{\{max\}}$ (e.g., 8, 16, or 32) allows, additional connections are spawned.
- Dynamic Bisection Routine:** When an active connection completes its assigned range and transitions to an idle state—or when a new connection socket is spun up:
 - The scheduler scans all currently active byte ranges: $R_i = e_i - p_i$, where e_i is the segment end boundary and p_i is the current read pointer of connection i .
 - It identifies the connection k possessing the maximum remaining unread byte workload: $k = \arg\max_i (e_i - p_i)$.
 - If $R_k > \text{Threshold}_{\{min\}}$ (typically > 512 KB to avoid excessive connection overhead), the scheduler dynamically bisects the remaining span:
$$M_k = p_k + \lfloor (e_k - p_k) / 2 \rfloor$$
 - Connection k 's endpoint is truncated: $e_k \leftarrow M_k$.
 - The newly available idle connection j is immediately dispatched with a range request:`Range: bytes=(M_k + 1)-e_k_{old}`
- Straggler Elimination:** If a particular connection slows down significantly due to server-side TCP window scaling penalties, the remaining unread bytes are continually split and offloaded to faster connections, ensuring 100% line saturation until the final byte is received.

2.2 Connection Pooling, Keep-Alive & Socket Tuning

Unlike naive downloaders that tear down TCP connections between chunks, IDM uses persistent HTTP keep-alive connections (`Connection: keep-alive`). When a segment finishes, its underlying socket is returned to an internal connection pool rather than issuing a TCP `FIN` packet. Subsequent byte ranges bypass the 3-way TCP handshake (`SYN` , `SYN-ACK` , `ACK`) and TLS 1.2/1.3 session renegotiations, eliminating hundreds of milliseconds of round-trip latency. Furthermore, socket receive buffers (`SO_RCVBUF`) are scaled to match the Bandwidth-Delay Product (BDP):

$$\text{BDP} = \text{Bandwidth} \times \text{RTT}$$

Architecture	Segmentation Type	Connection Management	Straggler / Bottleneck Vulnerability
Standard Browser / cURL	None (Single Stream)	Single TCP Socket	Extreme. Limited by single-stream TCP congestion window and server-side connection caps.
Static Segmenters (aria2 default)	Fixed Static Chunks (N parts)	New Socket per segment or static map	High. If chunk #3 is throttled by server QoS, total task waits for chunk #3 while others sit idle.
Internet Download Manager	Dynamic Bisection (Half-Split)	Pooled Keep-Alive Sockets (up to 32)	Zero. Faster connections dynamically steal unread bytes from lagging connections.
OmniFetch (Target Engine)	Dynamic Bisection + Sparse File	HTTP/2 & HTTP/3 (QUIC) Multiplexed	Zero. Eliminates both TCP head-of-line blocking and disk rebuilding I/O overhead.

3. Resumption, Fault Tolerance & Expired Link Refresh

Network disruptions, computer sleep states, dynamic IP reassignments, and CDN token expiries require a multi-tiered recovery engine. IDM's resilience is rooted in rigorous HTTP caching semantics and its proprietary URL hot-swapping routine.

3.1 Cache Validation & Remote Integrity Verification

When resuming a paused or interrupted download, IDM must guarantee that the remote resource has not been modified or replaced on the server. If the remote file has changed, continuing to download subsequent byte offsets and concatenating them with older chunks would result in severe file corruption:

- **ETag Verification:** The client stores the server's unique entity tag (`ETag: "686897696a7c876b7e"`). Upon resumption, IDM dispatches an HTTP request with `If-Match: [stored-ETag]` or `If-Range: [stored-ETag]` . If the ETag matches, the server returns `206 Partial Content` . If the ETag differs, the server returns `412 Precondition Failed` or `200 OK` with the full new file, prompting IDM to notify the user.
- **Last-Modified Fallback:** If ETags are absent, IDM checks `Last-Modified` using `If-Unmodified-Since` . Any timestamp disparity triggers a conflict-resolution workflow.
- **Segment State Journaling:** IDM serializes active segment metadata to disk in a transactional companion state file (mapping active connection boundaries, verified byte lengths, and CRC32 checkpoints of written data) updated every 500ms. An unexpected power outage only loses unwritten memory buffers.

3.2 The Expired Link Refresh Architecture ("Refresh Download Address")

Modern cloud file storage services (e.g., Google Drive, AWS S3 pre-signed URLs, CloudFront signed cookies, Rapidgator, Mega) enforce short-lived URLs containing cryptographic expiry timestamps (e.g., `?X-Amz-Expires=3600&X-Amz-Signature=...`). When an interrupted download is resumed after 2 hours, the server rejects requests with `HTTP/1.1 403 Forbidden` or `410 Gone` .

IDM'S "REFRESH DOWNLOAD ADDRESS" SUBSYSTEM WORKFLOW

1. When an active connection fails with 401, 403, or 410 during resumption, the task enters a suspended `STALLED_EXPIRED_LINK` state.
2. The user triggers "Refresh Download Address" (or IDM invokes it automatically).
3. IDM launches the default browser navigating back to the recorded `Referer` URL (the landing page where the user initially clicked the download).
4. IDM enters an *Interception Listening Mode* keyed to the download task ID.
5. When the user solves the CAPTCHA or clicks the new download button in the browser, the browser extension captures the freshly minted signed URL along with new session cookies.
6. Instead of initiating a new download task, the extension passes the new URL and cookies to `IDMan.exe` via Native Messaging with an existing task correlation flag.
7. IDM hot-swaps the underlying target URI, injects updated session headers, resets the connection pool, and executes HTTP Range requests starting precisely from the previously persisted byte offsets. Zero downloaded bytes are wasted!

4. Deep Browser Integration & Stream Interception

IDM's ubiquitous capture ability relies on a dual-tier interception architecture that catches both high-level browser actions and low-level system-wide network socket traffic.

4.1 Browser WebExtensions & Native Messaging Protocol

In modern browsers (Chrome, Firefox, Edge, Opera, Brave, Vivaldi), IDM installs the **IDM Integration Module**. The extension uses Chrome's `webRequest` and `downloads.onDeterminingFilename` APIs:

- **Interception Rules:** When a network response begins, the extension evaluates the HTTP response headers. If the `Content-Disposition` header specifies an attachment or the URL path ends with an extension matching IDM's internal registry (e.g., `ZIP`, `EXE`, `RAR`, `ISO`, `MP4`, `PDF`, `MSI`, `TAR.GZ`), the browser download is canceled immediately (`chrome.downloads.cancel()`).
- **Context Forwarding:** The extension packages the complete HTTP state:

```
{
  "url": "https://cdn.example.com/builds/v2.4.0.iso",
  "referrer": "https://example.com/downloads",
  "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)...",
  "cookies": "session_id=987abc; auth_token=xyz123...",
  "postData": null,
  "customHeaders": { "X-Requested-With": "XMLHttpRequest" }
}
```

This payload is serialized to JSON and pushed over standard I/O to `IDMMsgHost.exe`, which hands it off to `IDMan.exe` via Windows Named Pipes (`\\.\pipe\IDM_IPC_PIPE`).

- **Keyboard Interception Overrides:**
 - **Bypass Modifier:** Holding the ALT key during a link click suppresses IDM's interception hooks, forcing the browser to execute its native download manager.
 - **Force Modifier:** Holding the CTRL key forces IDM to capture and download any clicked hyperlink regardless of file extension or MIME type.

4.2 System-Wide Network Driver Interception (WFP & Winsock)

To capture downloads initiated by applications lacking browser extensions (e.g., legacy browsers, desktop clients, email software, command-line scripts), IDM utilizes a kernel-level **Windows Filtering Platform (WFP) Callout Driver** (`idmgr.sys`) and network API hooking (`IDMNetMon.dll`). The driver monitors TCP port 80 (HTTP) and port 443 (HTTPS) connection establishment, inspects outgoing HTTP request streams, and identifies binary content downloads by sniffing incoming response headers.

5. Streaming Media Interception (Video Grabber) Subsystem

One of IDM's most popular features is the floating **Video Download Panel** that appears above embedded media players across the web. The underlying capture engine handles modern adaptive streaming architectures.

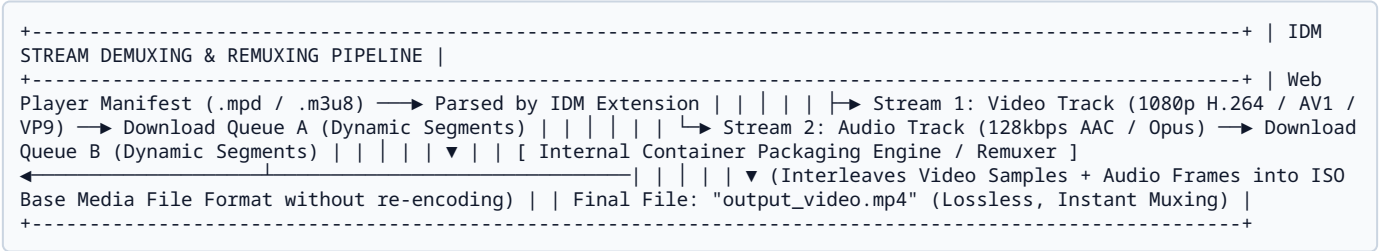
5.1 Manifest Sniffing & Segment Interception

Modern video streaming platforms rarely serve a monolithic MP4 file. Instead, they use adaptive bitrate streaming protocols:

- **HTTP Live Streaming (HLS):** IDM sniffs the network stream for `.m3u8` master playlists. It parses the master playlist to identify available resolution variants (e.g., 1080p, 720p, 480p) and target media playlists containing sequences of `.ts` (MPEG-2 Transport Stream) or `.fmp4` (Fragmented MP4) chunks.
- **MPEG-DASH:** IDM sniffs XML-based `.mpd` (Media Presentation Description) manifests. It parses the `<AdaptationSet>` elements to decouple independent video representations and audio representations.

5.2 Decoupled Stream Demuxing & Remuxing

Platforms like YouTube stream high-definition video (1080p, 1440p, 4K) and high-quality audio as completely separate HTTP streams to minimize bandwidth during adaptive bitrate shifts.



When the user clicks "Download 1080p", IDM spawns two synchronized sub-tasks: one downloading the isolated video stream and the other downloading the isolated audio stream. Once both streams complete, IDM invokes an internal media container remuxer (similar to FFmpeg's `-c copy`) that packetizes both bitstreams into a single continuous `.mp4` or `.mkv` file without quality loss or CPU-intensive transcoding.

5.3 Subtitle & DRM Constraints

IDM automatically scans for subtitle descriptors in manifests (WebVTT `.vtt` and SubRip `.srt`) and adds them as downloadable options in the panel. When streams are protected by DRM (e.g., Widevine DRM, PlayReady, FairPlay) via Encrypted Media Extensions (EME), IDM detects the Common Encryption (`cenc`) indicators and informs the user that the protected stream cannot be decrypted without legal content keys.

6. Site Grabber, Web Spider & Recursive Crawler Engine

The **IDM Site Grabber** is a full-featured web spider designed for batch asset harvesting, directory mirroring, and offline website preservation.

Grabber Module	Configuration Parameters	Technical Functionality
Crawl Depth & Traversal	Levels to explore (1 to N, or unlimited). Boundary scopes: Base directory only, Entire domain, Subdomains, or External domains.	Implements an asynchronous breadth-first search (BFS) crawler parsing HTML DOM nodes: <code><a href></code> , <code></code> , <code><video></code> , <code><script></code> , and CSS <code>url()</code> rules.
File Pattern Filtering	Inclusion/Exclusion file masks (e.g., <code>*.jpg;*.png;*.webp</code>). Min/Max file size thresholds. Regex URL filters.	Filters candidate URLs against MIME types and file extension tables. Ignores thumbnails/tracking pixels below size thresholds (e.g., skip images < 50 KB).
Exploration vs. Queuing	Two execution modes: 1. <i>Interactive Tree Exploration</i> 2. <i>Automated Queue Injection</i>	Interactive mode builds a real-time hierarchical GUI tree of discovered assets, allowing users to uncheck unwanted branches before downloading. Queue mode automatically batches files into the scheduler.
Offline Mirroring & Link Rewriting	Convert links for offline browsing option.	Rewrites absolute external hyperlinks within downloaded HTML/CSS files to relative local filesystem paths (<code>href="./assets/css/style.css"</code>), enabling offline browsing.
Periodic Site Synchronization	Scheduled synchronization intervals (Daily, Weekly, Hourly).	Re-crawls target sites periodically. Issues HTTP <code>HEAD</code> requests evaluating <code>If-Modified-Since</code>

Grabber Module	Configuration Parameters	Technical Functionality
		and <code>ETag</code> to only pull newly updated or modified files.

7. Queue Management, Scheduling & System Automation

IDM features a robust queuing and scheduling subsystem capable of unattended overnight batch processing and automated power management.

- **Multi-Queue Topology:** Users can create an unlimited number of isolated download queues (e.g., "Critical Work", "Torrents/ISOs", "Streaming Media", alongside the default "Main Queue" and "Synchronization Queue"). Each queue maintains its own concurrency limit, allowing users to enforce strict sequential downloading (1 file at a time) or parallel downloading (up to N concurrent downloads).
- **Temporal Cron-Style Scheduler:** Queues can be configured with discrete execution windows:
 - Start queue at exact timestamp (e.g., 02:00:00 AM to leverage ISP unmetered off-peak hours).
 - Stop queue at exact timestamp (e.g., 07:30:00 AM) or after a maximum elapsed duration.
 - Recurring daily/weekly schedules (e.g., execute only on Mondays, Wednesdays, and Fridays).
 - Automated retry policies: Configurable retry counters (e.g., retry failed items 5 times with exponential backoff).
- **Hardware & System Power Hooks:** Upon completion of all items in a queue, IDM can trigger automated OS operations:
 - **Modem / VPN Disconnect:** Issues Windows Remote Access Service (RAS) API calls to terminate dial-up or PPPoE connections.
 - **Process Exit:** Safely terminates `IDMan.exe` and flushes all catalog state caches.
 - **Power Management:** Issues Windows API calls (`SetSuspendState` / `InitiateSystemShutdownEx`) to enter Standby, Hibernate, or complete PC Shutdown. IDM displays a 30-second abort countdown dialog before initiating shutdown to prevent accidental data loss.
 - **ACPI Wake Timers:** Registers Windows system wake-up timers (`CreateWaitableTimer` with `SetWaitableTimer(..., fResume=TRUE)`) to automatically wake a sleeping PC when a scheduled queue is due to start.

8. Bandwidth Shaping, Speed Limiter & Quotas

To prevent download streams from choking other critical network applications (such as VoIP calls, online gaming, or video conferences), IDM provides precise traffic shaping.

8.1 Token Bucket Rate-Limiting Mechanics

IDM's **Speed Limiter** implements a classic **Token Bucket Algorithm** on the socket read loop:

- Tokens are added to a virtual bucket at a constant rate corresponding to the user-defined bandwidth ceiling *R* (e.g., 500 KB/s).
- The socket read thread reads from the TCP socket buffer in discrete chunks (`B_{chunk}`), typically 4 KB to 16 KB).
- Before each read, the thread checks the token bucket. If sufficient tokens are available, the bytes are read and the corresponding tokens are subtracted. If tokens are depleted, the worker thread yields execution via high-resolution sleep timers (`nanosleep` / `timeSetEvent`) until the bucket replenishes.
- **Granularity:** Rate limiting can be toggled globally across all active downloads or configured on an individual task level.

8.2 Dynamic Web Protection ("Smart Limiter") & Bandwidth Quotas

- **Smart Limiter Mode:** IDM detects foreground browser HTTP requests. When user browsing is detected, IDM dynamically throttles its background download sockets by 50–70%, preventing browser page load latency spikes, and restores full speed once browser activity ceases.
- **Fair Access Policy (FAP) / Bandwidth Quotas:** Designed for satellite and metered cellular connections. Users can set a hard limit (e.g., maximum 5,000 MB per 24 hours). Once the quota is consumed, all active queues pause automatically until the rolling quota window resets.

9. Network Protocols, Proxies, and Security

IDM's enterprise protocol support guarantees compatibility across diverse corporate and secure network infrastructures.

Category	Protocols & Standards Supported	Implementation Specifics
Transfer Protocols	HTTP/1.0, HTTP/1.1, HTTPS (TLS 1.2, TLS 1.3), FTP (Active & Passive PASV modes), FTPS (FTP over TLS/SSL), legacy MMS (Microsoft Media Server).	Supports chunked transfer encoding, dynamic byte ranges, custom HTTP verbs, and automatic FTP PASV/PORT fallback when behind strict NAT firewalls.
Enterprise Authentication	HTTP Basic Authentication, Digest Authentication, NTLM (v1/v2), and Kerberos / SPNEGO Negotiate.	Site Login Manager stores domain-specific credentials, automatically negotiating challenge-response handshakes without interrupting user workflows.
Proxy & VPN Routing	HTTP Proxy, HTTPS Proxy, SOCKS4, SOCKS4a, SOCKS5, Microsoft ISA Server, Forefront TMG. Proxy Auto-Configuration (PAC) scripts.	Evaluates JavaScript PAC files (<code>FindProxyForURL(url, host)</code>) dynamically. Supports proxy bypass lists, upstream proxy authentication, and system proxy auto-detection.
Antivirus Integration	Dynamic post-download invocation of external scanners (Windows Defender, Avast, Kaspersky, Bitdefender, AVG, etc.).	Executes the configured AV binary with path parameters upon task completion: "C:\\Program Files\\Antivirus\\scan.exe" "%file%" Monitors exit codes before declaring download safe.
Hash Integrity Checking	MD5, SHA-1, SHA-256 cryptographic checksum validation.	Computes streaming hashes during disk writes. Compares against server-supplied hashes or user-input checksums to verify zero byte corruption.

10. Storage Engine, Disk I/O & File Assembly

How IDM manages disk I/O during multi-part downloading represents one of its most critical architectural characteristics—and its greatest performance bottleneck.

10.1 The Temporary Directory & Segment Layout

For each active download task, IDM allocates a folder inside its configured temporary directory (e.g., `%APPDATA%\\IDM\\DwnlData\\[Username]\\...`). As dynamic segments are negotiated, IDM creates isolated partial files on disk:

```
DwnlData/Administrator/Build_ISO_10482/
├─ Build_ISO.log           (Binary state journal: chunk ranges, offsets, ETag)
├─ Build_ISO.tmp_0         (Bytes 0 - 26,214,400)
├─ Build_ISO.tmp_1         (Bytes 26,214,401 - 52,428,800)
├─ Build_ISO.tmp_2         (Bytes 52,428,801 - 78,643,200)
└─ Build_ISO.tmp_3         (Bytes 78,643,201 - 104,857,600)
```

10.2 The Rebuilding / Stitching Bottleneck

When all segments reach 100% completion, IDM enters its notorious **"Rebuilding / Assembling" phase**. It creates the final destination file (e.g., in `C:\\Users\\Downloads\\Programs\\Build_ISO.iso`) and sequentially reads each `.tmp_n` file, appending its bytes to the final file before deleting the temporary chunks:

- **Disk I/O Penalty:** This process forces a **2x write amplification**. A 10 GB file is written once to the temp directory during download, and then read and written a second time during rebuilding!
- **Mechanical & SSD Wear:** On mechanical HDDs or saturated SSDs, this causes severe latency spikes at 99%, often taking several minutes to stitch massive files. If the temp folder and destination folder reside on different physical drives, the entire file must be transferred across disk buses.

10.3 Automatic File Categorization, Drag-and-Drop, & ZIP Preview

- **Category-Based Folder Routing:** Automatically classifies downloads into specific subfolders based on extension:
 - *Compressed:* `.zip`, `.rar`, `.7z`, `.tar.gz`, `.iso`
 - *Documents:* `.pdf`, `.doc`, `.docx`, `.xls`, `.epub`, `.txt`
 - *Music:* `.mp3`, `.flac`, `.wav`, `.aac`, `.m4a`
 - *Programs:* `.exe`, `.msi`, `.apk`, `.dmg`, `.bin`
 - *Video:* `.mp4`, `.mkv`, `.avi`, `.mov`, `.webm`

- **Floating Drop Target ("Drop Basket"):** A translucent floating widget that stays on top of the desktop. Users can drag hyperlinks directly onto the widget to queue them immediately. Additionally, completed downloads in IDM's list view can be dragged directly into Windows Explorer, media players, or chat windows.
- **ZIP Preview Subsystem:** A powerful feature allowing users to view the contents of a remote `.zip` archive without downloading the full file. IDM sends an HTTP Range request for the final 64 KB of the file to fetch the ZIP *Central Directory record*, parses the file hierarchy, and lets users selectively download specific individual files!

11. Automation, Command Line Interface (CLI) & COM API

IDM exposes rich automation capabilities for scripts, batch files, third-party launchers, and enterprise software.

11.1 Complete Command Line Interface (`idman.exe`) Matrix

`idman.exe [/d URL] [/s] [/p local_path] [/f local_file_name] [/q] [/h] [/n] [/a]`

Parameter	Argument Syntax	Technical Description
/d URL	/d "https://domain.com/file.zip"	Designates the target URL to download. Core required parameter for link processing.
/s	/s	Triggers the scheduler to start executing the currently active download queue immediately.
/p local_path	/p "D:\Downloads\ISO"	Overrides default category storage directories; defines explicit local destination folder.
/f file_name	/f "Target_Image.iso"	Overrides server-suggested filename; saves the downloaded resource under this explicit filename.
/q	/q	Commands IDM to terminate its process (<code>IDMan.exe</code>) immediately once downloading finishes.
/h	/h	Commands IDM to hang up the active dial-up/VPN connection upon successful download completion.
/n	/n	Enables Silent Mode . Suppresses all GUI dialogs, prompts, and confirmation boxes.
/a	/a	Enqueues the specified file (<code>/d</code>) into the download queue without starting it immediately.

11.2 Batch Export/Import Formats & COM Automation

- **Batch Export Format (`.ef2`):** IDM supports exporting queue lists into text-based `.ef2` manifest files containing URL endpoints, cookies, destination paths, and authentication parameters for batch migration across machines.
- **Windows COM / ActiveX Interface:** IDM registers COM interfaces (such as `CIDMLinkTransmitter` , CLSID: `{ECF74409-C4E0-4788-83DA-1C6D0D2E46D1}`) in the Windows Registry. Any C++, C#, Python, or Delphi application can instantiate this COM object to programmatically inject downloads with full cookie jars and referrers directly into IDM's runtime memory without spawning CLI sub-processes.

12. The OmniFetch Engineering Blueprint: Surpassing IDM

While IDM remains a benchmark for download acceleration, its codebase is encumbered by legacy Win32 design choices dating back to the early 2000s. The table below outlines how **OmniFetch** can adopt IDM's proven strengths while leapfrogging its architecture using modern systems engineering.

Subsystem	IDM Legacy Implementation (Bottlenecks)	OmniFetch Modern Architecture (Recommended)
Disk Storage & Assembly	Writes isolated <code>.tmp_n</code> files. Requires a CPU- and I/O-heavy "Rebuilding" phase that concatenates chunks, causing a 2x write penalty and 99% stalls.	Zero-Stitch Sparse File Allocation: Pre-allocate the entire file instantly using <code>fallocate()</code> (Linux/macOS) or <code>FSCTL_SET_SPARSE / SetFileValidData()</code> (Windows). Write incoming chunk buffers directly to their

Subsystem	IDM Legacy Implementation (Bottlenecks)	OmniFetch Modern Architecture (Recommended)
		exact byte offsets via <code>pwrite()</code> . Assembly time is 0.00 seconds!
Transport Protocols	Primarily HTTP/1.1 multi-socket pooling. Vulnerable to TCP Head-of-Line blocking, TLS renegotiation latency, and connection churn.	Native HTTP/3 (QUIC) & HTTP/2: Single UDP socket multiplexes multiple streams with zero Head-of-Line blocking, 0-RTT connection resumption, and seamless connection migration across network changes (e.g., Wi-Fi to 5G).
Async I/O Runtime	Legacy Win32 synchronous multi-threading / worker thread pool per connection. Scalability caps around 32 threads.	High-Performance Async Engine: Built with Rust (Tokio) or Go utilizing Linux <code>io_uring</code> and Windows I/O Completion Ports (IOCP) . Capable of saturating 10 Gbps+ fiber interfaces with minimal CPU/RAM overhead.
Video & Media Extraction	Proprietary closed-source regex sniffers. Frequently breaks when streaming sites alter their player code; requires continuous manual patch updates.	Integrated Modular Extractor Engine: Embed an internal, auto-updating yt-dlp core with native C-bindings to FFmpeg (libavformat / libavcodec) for in-process demuxing and loss-less remuxing across 1,000+ streaming platforms.
Protocol Versatility	Restricted to client-server protocols (HTTP, HTTPS, FTP). Cannot download peer-to-peer torrents.	Hybrid Protocol Engine: Unified queue seamlessly handling HTTP/3, HTTPS, SFTP, and BitTorrent / WebTorrent within a single interface and shared bandwidth limiter.
Platform Portability & UI	Windows-exclusive 32-bit Win32 GUI. Rigid, outdated aesthetic with no official macOS or Linux support.	Cross-Platform Modern Architecture: Modular headless daemon (<code>omnifetchd</code>) with lightweight, GPU-accelerated UI (via Tauri + React/Svelte or Flutter) running natively across Windows 11, macOS (Apple Silicon), and Linux.

SUMMARY: OMNIFETCH CORE IMPLEMENTATION ROADMAP

- **Phase 1 (Core Engine):** Implement asynchronous dynamic bisection segmentation in Rust/Go with sparse file pre-allocation to eliminate the rebuilding bottleneck entirely.
- **Phase 2 (Browser Integration):** Build Manifest V3 WebExtensions for Chrome, Edge, and Firefox communicating with OmniFetch via Native Messaging, featuring context forwarding and hot-swappable expired link recovery.
- **Phase 3 (Streaming Media):** Embed modular HLS/DASH manifest parsers and FFmpeg remuxing pipelines with floating DOM video download overlays.
- **Phase 4 (Automation & Traffic Shaping):** Implement multi-queue cron schedulers, Token Bucket rate limiters, PAC proxy parsers, and a comprehensive CLI / REST API.